Теоретическая информатика, осень 2026 г. Лекция 4. Алгоритм минимизации DFA. Теорема Майхилла–Нероуда. Двусторонние конечные автоматы (2DFA). Преобразование 2DFA к DFA*

Александр Охотин

6 октября 2026 г.

Содержание

1 Алгоритм минимизации DFA **1**

2 Классы эквивалентности, определяемые регулярным языком **6**

3 Двусторонние конечные автоматы (2DFA) **7**

4 Преобразование 2DFA к DFA **9**

1 Алгоритм минимизации DFA

Понятие языка строк, принимаемых детерминированным конечным автоматом (DFA), начиная из состояния q, определяет отношение эквивалентности на множестве состояний: два состояния q и q' *эквивалентны*, если из них принимаются одни и те же строки. Если же множества $L_{\mathcal{A}}(q)$ и $L_{\mathcal{A}}(q')$ не совпадают, то существует строка, принадлежащая одному из них и не принадлежащая другому — *разделяющая строка*.

Алгоритм Мура [1956] для минимизации автоматов находит отношение эквивалентности на множестве Q, измельчая классы эквивалентности. Это делается путём определения некоторых неэквивалентных пар состояний, из-за которых имеющиеся классы необходимо разбить.

Сперва, если одно состояние — принимающее, а другое — отвергающее, то они, очевидно, неэквивалентны (ведь из одного ε принимается, а из другого отвергается — стало быть, ε — разделяющая строка). Поэтому в качестве начального разбиения берутся два класса эквивалентности: $R_0 = Q \setminus F$, $R_1 = F$.

Алгоритм поддерживает следующий инвариант: для текущего разбиения Q на непересекающиеся классы $R_0, \ldots, R_{m-1}$ верно, что для каждой пары классов R_i, R_j существует строка w, которая принимается из всех состояний одного класса и ни из одного из состояний другого класса. Такая строка нигде не хранится, но её существование обеспечивается.

На каждом шаге работы алгоритма просматриваются все имеющиеся классы, и для каждого класса R и для всякого символа $a \in \Sigma$, рассматриваются переходы из каждого

*Материалы к лекциям, прочитанным студентам 2-го курса факультета МКН СПбГУ в осеннем семестре 2026–2027 учебного года. Страница курса: `http://users.math-cs.spbu.ru/~okhotin/teaching/tcs_2026/`.

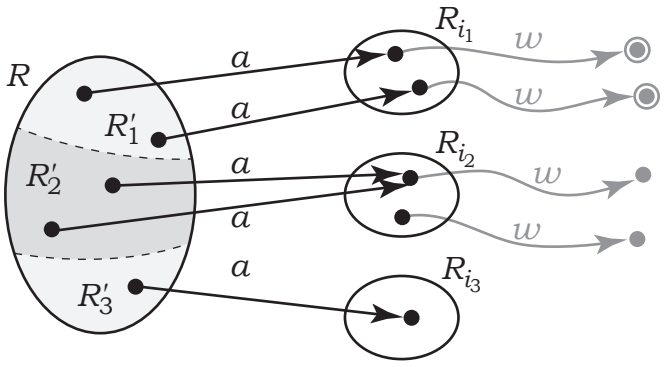


Рис. 1: Разбиение класса R на три класса на основе переходов по символу a.

состояния множества R по a. Если все эти переходы ведут в состояния из одного и того же класса, то никаких противоречий не обнаружено, и оснований разбивать класс R нет. Если же, однако, по a из R можно перейти в разные классы, то пусть $R = R'_1 \cup \ldots \cup R'_k$, где все переходы из R'_j по a идут в один из существующих классов R_{i_j}. Тогда R заменяется на классы $R'_1, \ldots, R'_k$, как показано на рис. 1. Действительно, если классы R_{i_1} и R_{i_2} различаются по некоторой строке w, то R'_1 и R'_2 различаются по строке aw.

Алгоритм 1 Минимизация DFA, алгоритм Мура [1956]

Дано: DFA $(\Sigma, Q, q_0, \delta, F)$.
В каждый момент времени алгоритм помнит разбиение Q на классы эквивалентности $R_0, \ldots, R_{m-1}$.

```
Удалить из исходного автомата все недостижимые состояния
Пусть R_0 = Q \ F, R_1 = F
while можно разделить какой-то класс do
    for all класс R do
        for all символ a ∈ Σ do
            for all q ∈ R do
                Определить класс, в котором находится δ(q, a)
            if классы для разных q не все совпадают then
                Пусть R = R'_1 ∪ ... ∪ R'_k, где переходы по a из каждого R'_j идут
                в один класс, а переходы из разных R'_j идут в разные классы
                Заменить класс R на классы R'_1, ..., R'_k
```

Пусть $R_0, \ldots, R_{m-1}$ — полученные классы. Алгоритм выдаёт DFA $(\Sigma, \{R_0, \ldots, R_{m-1}\}, [q_0], \delta', \{\, R_i \mid R_i \subseteq F \,\})$, где $[q_0]$ — это класс, в который попало состояние q_0, а $\delta'(R_i, a) = [\delta(r, a)]$ для $r \in R_i$ — класс, в который ведут все переходы по a из состояний из R_i.

Это продолжается, пока дальнейшее измельчение классов возможно. Когда далее измельчать становится некуда, строится новый автомат, состояниями которого станут построенные классы эквивалентности.

Теорема 1 (Мур [1956]). *Алгоритм 1 строит автомат, распознающий тот же язык, что и исходный, и минимальный среди всех автоматов, распознающих этот язык. Алгоритм работает за время $O(|\Sigma| \cdot n^2)$, где n — число состояний в исходном автомате, а Σ — алфавит.*

Доказательство. Алгоритм минимизации DFA (алгоритм 1) начинает свою работу с удаления всех состояний, не достижимых из начального. Это делается за линейное время любым алгоритмом поиска в графе. Далее алгоритм работает с автоматом, в котором для каждого состояния есть строка, по которой автомат придёт в это состояние из начального. Берётся начальное разбиение множества состояний на два класса эквивалентности, это разбиение измельчается, пока возможно, после чего строится новый автомат.

(тот же язык) Обозначение: $[q]$ — класс эквивалентности, в который попало состояние q. Поскольку алгоритм завершил измельчение классов, выполняется условие, что если $\delta(p, a) = r$, то $\delta'([p], a) = [r]$. Тогда, для всякой входной строки, всякому вычислению исходного автомата соответствует вычисление нового, проходящее через соответствующие классы эквивалентности.

$$q_0, q_1, q_2, \dots$$
$$[q_0], [q_1], [q_2], \dots$$

Поскольку каждый класс состоит целиком из принимающих или целиком из отвергающих состояний, в конце обоих вычислений выдаётся одно и то же решение. Отсюда следует, что строка w принимается исходным автоматом тогда и только тогда, когда её принимает новый автомат.

(минимальность) Пусть p и q — состояния исходного DFA, попадающие в разные классы эквивалентности. Утверждается, что есть строка, принимаемая из одного из них и отвергаемая из другого. Индукция по номеру шага алгоритма, на котором эти два состояния будут распределены в различные классы.

Базис: в разных классах с самого начала. Тогда одно из этих состояний принимающее, а другое — отвергающее, и разделяющая строка — ε.

Переход. Пусть на каком-то шаге оба состояния p, q принадлежат некоторому классу R, рассматриваются переходы из R по a, и выясняется, что $[\delta(p, a)] \neq [\delta(q, a)]$. Тогда состояния $\delta(p, a)$ и $\delta(q, a)$ попали в различные классы ещё прежде, и потому, по предположению индукции, есть строка w, принимаемая из одного из них и не принимаемая из другого. Тогда строка aw разделяет p и q.

Следовательно, из каждых двух состояний построенного автомата принимаются разные языки. Поскольку все состояния исходного автомата достижимы из начального, в построенном автомате каждое состояние достигается по некоторой строке. Отсюда, согласно лемме из предыдущего раздела, в любом автомате, распознающем этот же язык, по этим строкам должны достигаться различные состояния, что и доказывает минимальность построенного автомата.

(время работы) Сложность алгоритма оценивается так. На каждой итерации внешнего цикла в строчке 3 (кроме последней итерации) хотя бы один класс эквивалентности разделяется, и, соответственно, увеличивается количество классов. Так как классов никогда не станет больше, чем n, число итераций также ограничено числом n. Отсюда также следует, что строчки 9 и 10 будут выполнены не более чем n раз за всё время работы алгоритма. Каждое их выполнение потребует $O(n)$ действий, так что общее время выполнения этих строчек — $O(n^2)$.

На каждой итерации происходят следующие вычисления. Для каждого символа алфавита алгоритм просматривает каждое состояние исходного автомата один раз, в каком бы классе оно ни лежало — и потому строчка 7 будет выполнена $\Sigma \cdot n$ раз. Каждое её выполнение требует константного времени. Поэтому за всё время работы алгоритма эта строчка будет выполняться не более чем $|\Sigma| \cdot n^2$ раз и потребует $O(|\Sigma| \cdot n^2)$ действий. □

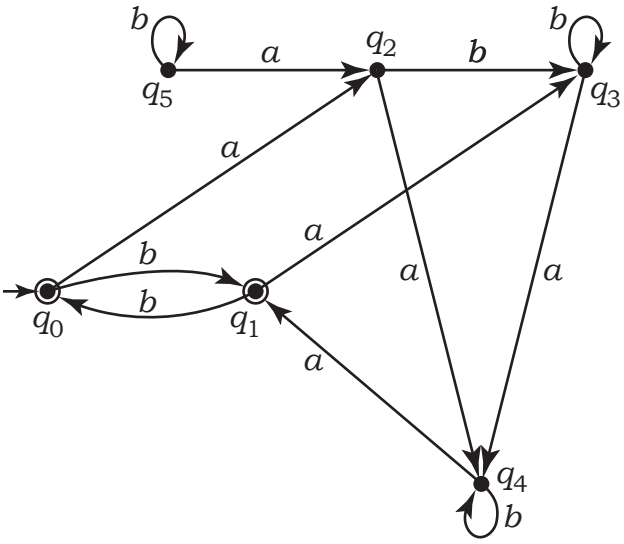


Рис. 2: Неминимальный DFA из примера 1.

Пример 1. *Автомат с 6 состояниями, приведённый на рис. 2, распознаёт язык* $\{ w \mid w \in \{a,b\}^*, |w|_a \equiv 0 \pmod 3 \}$, *хотя с виду этого не скажешь — надо долго всматриваться. Но стоит его минимизировать, как всё становится ясно.*

Сперва удаляется недостижимое состояние q_5.

Алгоритм начинает с разбиения $R_0 = \{q_2, q_3, q_4\}$, $R_1 = \{q_0, q_1\}$, *приведённого на рис. 3(левом верхнем). Класс* R_1 *разбить не удаётся ни по* a, *ни по* b, *а вот класс* R_0 *разбивается по* a, *поскольку из состояний* q_2 *и* q_3 *переходы по* a *ведут в* R_0 *же, а переход по* a *из состояния* q_4 *уходит в* R_1. *Поэтому* R_0 *заменяется на* $R_2 = \{q_2, q_3\}$ *и* $R_3 = \{q_4\}$, *полученное разбиение приведено на рис. 3(правом верхнем). Ничего больше разбить не удаётся, так что получается минимальный автомат с 3 состояниями, изображённый на рис. 3(нижнем).*

Как показали Бассино и др. [2012], если случайно выбрать автомат с n состояниями, то математическое ожидание времени работы алгоритма 1 будет порядка $n \log n$.

Известен более быстрый алгоритм минимизации DFA — алгоритм Хопкрофта [1971], работающий за время $O(|\Sigma| \cdot n \log n)$. Алгоритм Хопкрофта точно так же последовательно расщепляет классы эквивалентности, но за счёт хитроумных структур данных он меньше тратит времени на просмотр классов, которые расщепить не получается или пока не получается.

Для задачи минимизации NFA полиномиальный по времени алгоритм неизвестен. Вычислительная сложность этой задачи будет рассмотрена далее в курсе.

Применение минимизации DFA к распознаванию свойств автоматов. Как проверить для двух DFA, распознают ли они один и тот же язык? Сперва минимизировать оба. Если вышло разное число состояний, то языки разные. Иначе проверить два минимальных автомата на изоморфизм; а именно, попытаться построить взаимно однозначное соответствие между множествами состояний, начиная с начальных состояний и далее следуя функциям переходов — это делается за линейное время.

Другая задача: проверить, задаёт ли данное регулярное выражение язык Σ^*. Перевести его в DFA с экспоненциальным ростом, минимизировать, и затем проверить, получился ли

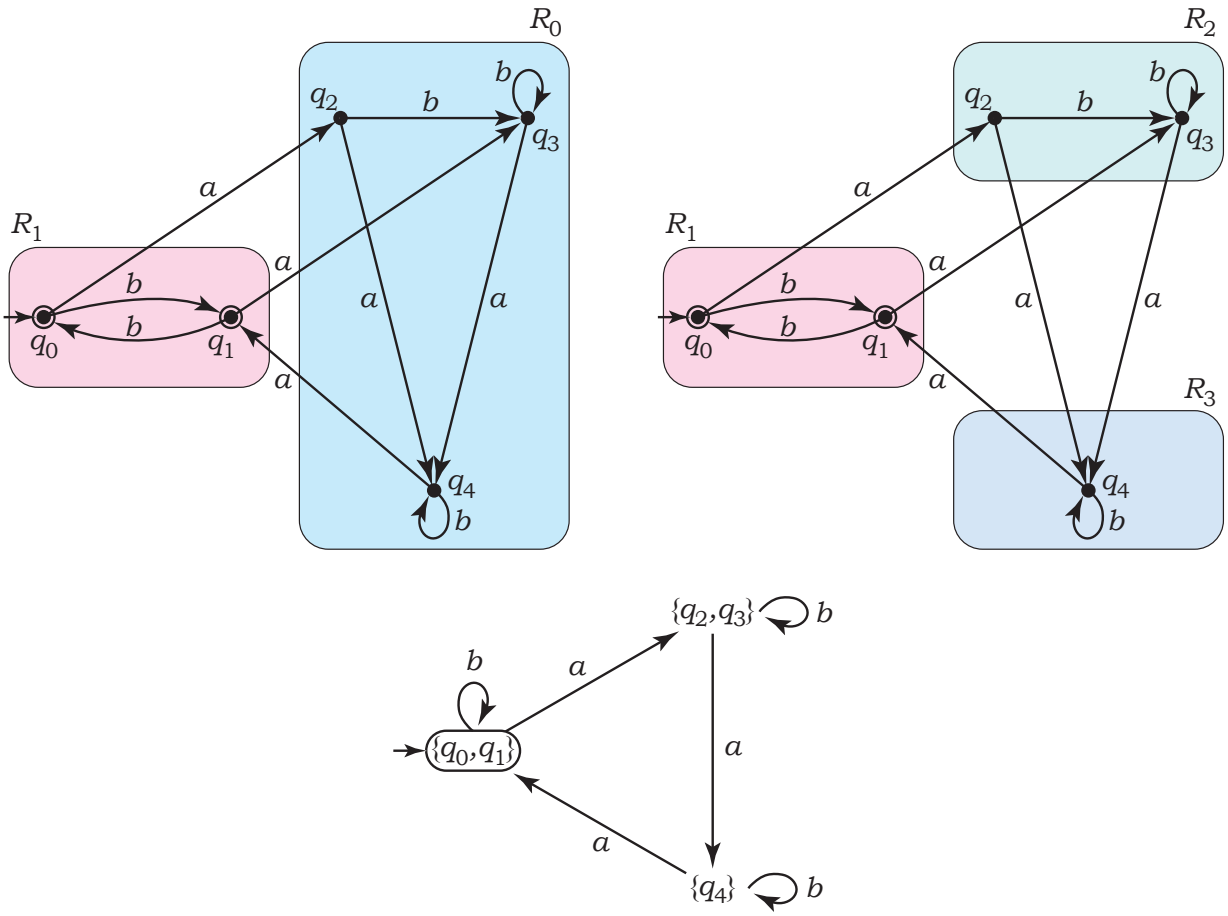


Рис. 3: Минимизация автомата из примера 1: (сверху слева) начальное разбиение; (сверху справа) измельчённое разбиение; (снизу) соответствующий минимальный DFA.




Рис. 4: Эдвард Мур (1925–2003); Джон Хопкрофт (род. 1939).

автомат с одним принимающим состоянием. Далее в курсе будет показано, что существенно более эффективных алгоритмов для решения этой задачи человечество не знает.

Упражнение 1. *Реализовать алгоритм Мура.*

Можно, например, завести два массива. В одном, индексируемом по номеру состояния, хранить номер класса, в котором находится это состояние. В другом массиве хранить перестановку номеров состояний — так, чтобы лежащие в одном классе состояния перечислялись одно за другим в любом порядке. При разделении класса удобно разделять его на два: множество состояний, переход из которых по a ведёт в какой-то определённый класс, и все остальные состояния — тогда номера этих состояний во втором массиве достаточно будет переставлять местами, как в процедуре разделения в алгоритме «быстрой сортировки».

2 Классы эквивалентности, определяемые регулярным языком

Доказательство правильности приводившегося ранее алгоритма минимизации DFA основывалось на рассуждениях о разбиении множества всех строк на классы эквивалентности, соответствующие состояниям автомата. Эти классы на самом деле можно определить и без автомата, основываясь только на языке, и все соответствующие рассуждения удобно представить в виде следующего абстрактного результата.

Теорема 2 (теорема Майхилла–Нероуда, см. Нероуд [1958])**.** *Пусть $L \subseteq \Sigma^*$ — язык, и пусть $\equiv_L$ — отношение эквивалентности на множестве Σ^*, где $u \equiv_L u'$, если для всякой строки v строки uv, $u'v$ или обе лежат в L или обе не лежат. Утверждается, что язык L регулярен тогда и только тогда, когда число классов эквивалентности в отношении $\equiv_L$ конечно.*

Доказательство. $\Rightarrow$ Если язык L регулярен, то для него есть минимальный DFA. Каждое состояние этого автомата становится классом эквивалентности, состоящим из всех строк, после чтения которых минимальный DFA приходит в данное состояние.

$\Leftarrow$ Обозначение: $[u]$ — класс эквивалентности, в который попала строка u.

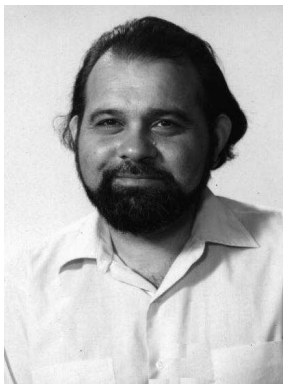



Рис. 5: Джон Майхилл (1923–1987), Анил Нероуд (род. 1932).

Справедливо следующее свойство: $u \equiv_L u'$ для некоторых строк $u, u' \in \Sigma^*$, то тогда $ua \equiv_L u'a$ для всякого символа $a \in \Sigma$. Действительно, для любого продолжения $w \in \Sigma^*$ строки uaw и $u'aw$ или обе лежат в L, или обе не лежат — это следует из эквивалентности u и u' для продолжения aw. Также потребуется второе свойство: всякий класс эквивалентности или целиком содержится в языке L, или не пересекается с ним; это верно потому, что строка из L не может быть эквивалентной строке не из L, так как для продолжения ε не выполнится условие.

Строится DFA, состояниями которого станут классы эквивалентности. Начальный класс — $q_0 = [\varepsilon]$. Переход из класса $S \subseteq \Sigma^*$ по символу $a \in \Sigma$ определяется как $\delta(S, a) = [ua]$, где u — произвольная строка из S; это определение не зависит от выбора строки u, потому что из $[u] = [u']$ по приведённому выше свойству следует $[ua] = [u'a]$. Наконец, класс $S \subseteq \Sigma^*$ — принимающее состояние построенного DFA, если $S \subseteq L$.

Утверждается, что, прочитав строку u, построенный DFA приходит в класс S, содержащий эту строку. Доказательство — индукция по длине u.

Базовый случай. Начальное состояние, $[\varepsilon]$, содержит строку ε.

Переход. Если автомат пришёл в класс S по строке u, то известно, что $u \in S$. Тогда переход по a ведёт в состояние $[ua]$, которое по определению содержит строку ua.

Наконец, пусть автомат прочитал всю входную строку w и пришёл в состояние S, содержащее w. Тогда, если $w \in L$, то $S \subseteq L$, и потому это состояние будет принимающим. Если же $w \notin L$, то $S \cap L = \varnothing$, и состояние, соответственно, окажется отвергающим. □

Рассуждение в приведённом доказательстве — то же, что и в доказательстве правильности алгоритма минимизации. Когда такое же рассуждение требуется в более сложном контексте, оказывает весьма полезным не повторять его заново, а использовать теорему Майхилла–Нероуда.

3 Двусторонние конечные автоматы (2DFA)

Двусторонние конечные автоматы — это автоматы, которые могут двигаться по входной строке налево и направо. Направление движения на каждом шаге задаётся функцией переходов. Функция переходов — частичная, по неопределённому переходу автомат отвергает. Входная строка ограничена с обеих сторон метками начала и конца, и на метке конца автомат может принять. Также автомат может зациклиться (то есть не остановиться) — и это всё равно что отвергнуть.

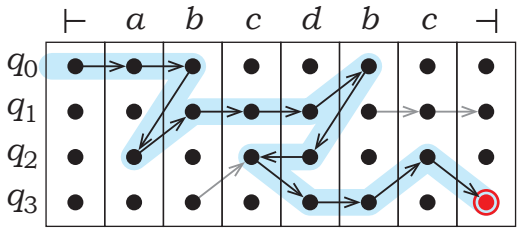


Рис. 6: Принимающее вычисление 2DFA.

Двусторонние автоматы также можно рассматривать как частный случай машины Тьюринга, которая не может изменять содержимого своей ленты — но удобнее следующее определение, обобщающее определение DFA.

Определение 1 (Рабин и Скотт [1959]). *Двусторонний детерминированный конечный автомат (2DFA: two-way deterministic finite automaton) — это пятёрка* $\mathcal{A} = (\Sigma, Q, q_0, \delta, F)$, *состоящая из следующих компонентов.*

- Σ — *алфавит, не содержащий двух особых символов* $\vdash, \dashv \notin \Sigma$ *(меток начала и конца строки).*
- Q — *конечное множество состояний.*
- $q_0 \in Q$ — *начальное состояние.*
- $\delta\colon Q \times (\Sigma \cup \{\vdash, \dashv\}) \to Q \times \{-1, +1\}$ — *частичная функция переходов.*
- $F \subseteq Q$ — *множество* ***принимающих состояний****, применяемых на правой метке конца* $(\dashv)$.

Для входной строки $w = a_1 \dots a_\ell \in \Sigma^*$, *пусть* $a_0 = \vdash$ *и* $a_{\ell+1} = \dashv$. *Вычисление 2DFA на* w — *это самая длинная, возможно бесконечная, последовательность* $(p_0, i_0), \dots, (p_j, i_j), \dots$, *где:*

- $p_j \in Q$ *и* $0 \leqslant i_j \leqslant \ell + 1$ *на каждом* j*-м шаге;*
- *начальная конфигурация* — $(p_0, i_0) = (q_0, 0)$;
- *всякая следующая конфигурация* (p_j, i_j), *если она определена, удовлетворяет* $\delta(p_{j-1}, a_{i_{j-1}}) = (p_j, d_j)$ *и* $i_j = i_{j-1} + d_j$.

Вычисление всегда определено единственным образом. Строка w ***принимается****, если вычисление конечно и его последняя конфигурация —* $(p_f, \ell{+}1)$, *для некоторого* $p_f \in F$. *Также вычисление может заканчиваться неопределённым переходом (ведь* δ — частичная *функция) или же продолжаться бесконечно; и в том и в другом случае* $\mathcal{A}$ *не принимает* w.

Язык, распознаваемый автоматом — $L(\mathcal{A}) = \{\, w \mid \mathcal{A}$ *принимает* $w \,\}$.

Вычисление 2DFA на данной строке w удобно представлять в виде графа, множество вершин которого, $V = Q \times \{0, 1, \dots, |w|, |w| + 1\}$ — это множество конфигураций автомата, и из каждой вершины исходит не более одной дуги, соответствующей переходу в этой конфигурации. Тогда вычисление — это путь, начинающийся в вершине $(q_0, 0)$. На рис. 6 показан пример принимающего вычисления.

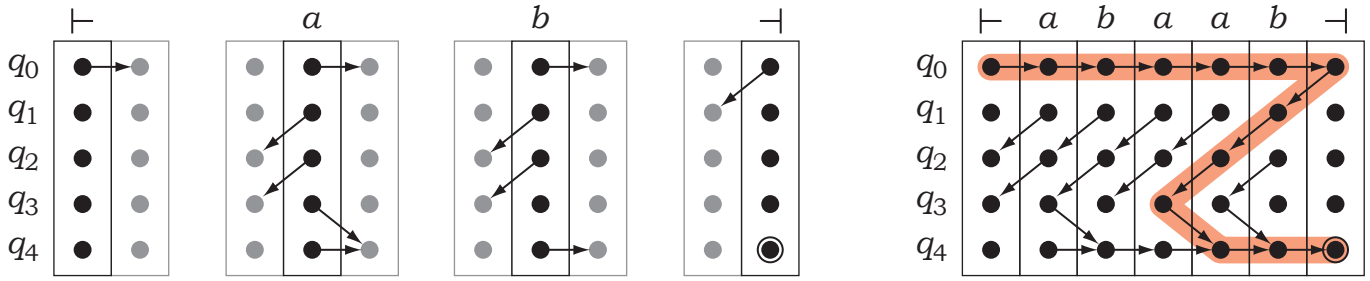


Рис. 7: 2DFA из примера 2: переходы по символам и по меткам начала и конца; вычисление на строке $w = abaab$, а также переходы из всех возможных конфигураций.

Пример 2. *Язык* $(a \mid b)^* a (a \mid b)^{n-1}$ *распознаётся 2DFA со множеством состояний* $Q = \{q_0, q_1, \ldots, q_n, q_{n+1}\}$. *Сперва автомат едет в конец в состоянии* q_0.

$$\delta(q_0, \vdash) = \delta(q_0, a) = \delta(q_0, b) = (q_0, +1)$$

Затем, доехав до метки конца, он разворачивается и отсчитывает n *символов.*

$$\begin{aligned} \delta(q_0, \dashv) &= (q_1, -1) \\ \delta(q_i, a) = \delta(q_i, b) &= (q_{i+1}, -1) \qquad (i \in \{1, \ldots, n-1\}) \end{aligned}$$

Если n*-й символ с конца окажется символом* a*, то автомат вновь отправится направо к метке конца, чтобы там принять.*

$$\begin{aligned} \delta(q_n, a) &= (q_{n+1}, +1) \\ \delta(q_{n+1}, a) = \delta(q_{n+1}, b) &= (q_{n+1}, +1) \end{aligned}$$

Множество принимающих состояний — $F = \{q_{n+1}\}$.

Если строка — не требуемого вида, то в ходе описанного вычисления автомат натолкнётся на неопределённый переход.

Переходы по $\vdash$, a, b *и* $\dashv$ *в автомате, построенном для* $n = 3$, *показаны на рис. 7. Также приведено принимающее вычисление автомата на строке* $w = abaab$, *в котором указаны переходы из всех возможных конфигураций на этой входной строке.*

4 Преобразование 2DFA к DFA

Оказывается, что способность ходить по ленте туда и обратно не наделяет двусторонний автомат способностью распознать какой-либо нерегулярный язык.

Теорема 3 (Рабин и Скотт [1959]; Шепердсон [1959])**.** *Для всякого 2DFA существует DFA, распознающий тот же язык.*

Лемма 1 (Шепердсон [1959]; Капуцис [2005])**.** *Для всякого 2DFA с* n *состояниями существует DFA с* $n(n^n - (n-1)^n) + 1$ *состояниями, распознающий тот же язык.*





Рис. 8: Джон Шепердсон (1926–2015) и Христос Капуцис (род. 1974).

Шепердсон [1959] не заботился об оптимальности построения и задействовал $(n+1)^{n+1}$ состояний; Капуцис [2005] впервые получил оптимальное построение — указанное число состояний в худшем случае нельзя уменьшить.

Доказательство. Выехав с метки начала ($\vdash$), 2DFA какое-то время движется направо; ничего не мешает DFA точно так же повторить его вычисление. Но потом 2DFA может развернуться и пойти налево, вновь обследуя символы, которые DFA уже прочитал и забыл. Как одностороннему автомату проследить это вычисление? Если знать наперёд, в каких состояниях 2DFA поедет назад, то DFA мог бы заранее проследить его обратный путь, читая строку в первый и единственный раз. Но узнать это заранее неоткуда — тем более, что 2DFA может сновать взад-вперёд по одним и тем же символам по сложной траектории. С другой стороны, для DFA неважно, какие именно зигзаги 2DFA выписывал по уже прочитанным символам — важно, вернётся ли 2DFA когда-нибудь к символу, который DFA только что прочитал, и перейдёт ли с него направо на следующий символ; и если это случится, важно знать, в каком состоянии он туда перейдёт.

Чтобы всё это проследить, DFA, прочитав строку u, будет помнить исход *всех возможных вычислений 2DFA на $\vdash u$, начинающихся в начальной конфигурации или на последнем символе строки u, и заканчивающихся выходом за пределы u направо.* Начальная конфигурация единственна, конфигураций на последнем символе конечное число, и исходы всех таких вычислений записываются в начальном состоянии построенного DFA. Далее, на каждом шаге DFA использует сведения о просчитанных вычислениях 2DFA на более короткой строке, находящиеся в состоянии DFA, для определения исхода вычислений на продолжении этой строки. Тогда к моменту прочтения всей входной строки DFA вычислит, в числе прочего, итог искомого вычисления, начинающегося в начальной конфигурации.

Пусть $(\Sigma, Q, \delta, q_0, F)$ — 2DFA. Состояния DFA имеют вид $(\widehat{q}, f)$, где $\widehat{q} \in Q$ — состояние 2DFA, а $f\colon Q \to Q$ — частичная функция, отображающая состояния в состояния. [1]. Автомат строится так, чтобы, прочитав всякую входную строку $u \in \Sigma^*$, он приходил бы в состояние $(\widehat{q}, f)$, где

I. $\widehat{q}$ — это то состояние, в котором 2DFA, начав работу на $\vdash u$ в начальной конфигурации, впервые перейдёт из самого правого символа $\vdash u$ направо;

II. если 2DFA начинает своё вычисление на самом правом символе $\vdash u$ в некотором состоянии q, то он со временем переходит с этого самого правого символа направо в

[1] Идея использования такой функции принадлежит Шепердсону [1959], но лишь Капуцис [2005] определил необходимое количество этих функций, а также их области определения и значений.

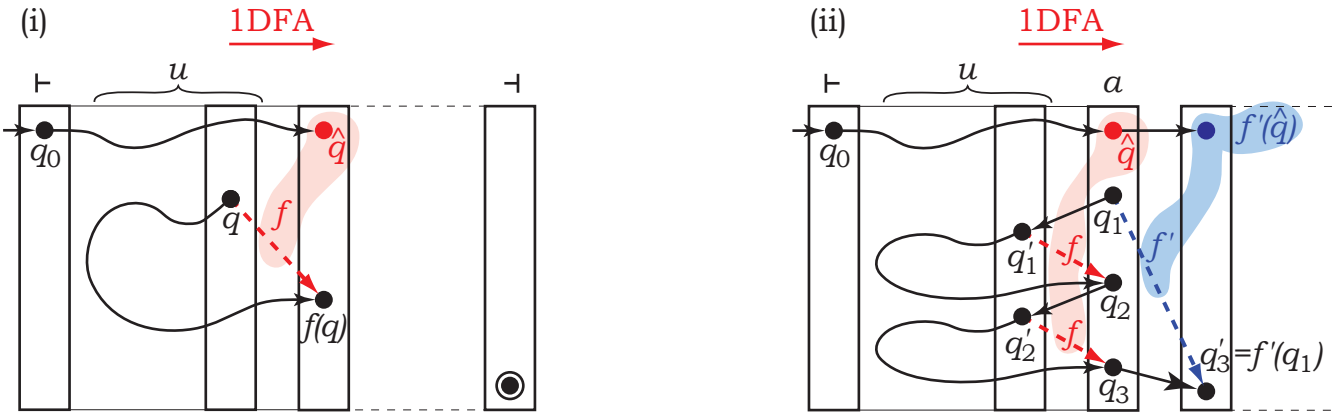


Рис. 9: Доказательство леммы 1: (i) сведения о поведении 2DFA на начальном участке строки, которые DFA запоминает в своём состоянии $(\widehat{q}, f)$; (ii) как DFA преобразует сведения для строки u в подобные же сведения для строки ua при переходе по символу a.

состоянии $f(q)$; если же это вычисление 2DFA отвергает или зацикливается, то значение $f(q)$ не определено: эта функция описывает *поведение* 2DFA на этом начальном участке строки.

Также в DFA есть особое мёртвое состояние, в которое он приходит по таким строкам u, что 2DFA, начав работу на $\vdash u$ в начальной конфигурации, *никогда* не перейдёт из самого правого символа $\vdash u$ направо.

Условия, определяющие состояние $(\widehat{q}, f)$, показаны на рис. 9(i).

Остаётся определить DFA, вычисляющий эти данные. Его начальное состояние — пара (q'_0, f_0), где q'_0 определяется переходом $\delta(q_0, \vdash) = (q'_0, +1)$ (по этому переходу автомат впервые выйдет направо из пустой строки) а функция f_0 записывает все переходы по метке начала.

$$f_0(q) = \begin{cases} q', & \text{если } \delta(q, \vdash) = (q', +1) \\ \text{не определено}, & \text{в противном случае} \end{cases}$$

Определение функции переходов показано на рис. 9(ii). Пусть DFA, прочитав некоторый префикс входной строки, находится в состоянии $(\widehat{q}, f)$ — то есть, вычислил сведения о вычислениях 2DFA на прочитанном префиксе, удовлетворяющие условиям I–II. Тогда на следующем шаге DFA читает следующий символ $a \in \Sigma$ и должен вычислить такие же сведения о вычислениях 2DFA на строке, продлённой этим символом. Определение функции переходов, разумеется, не зависит от входной строки, только от состояния и символа — но при построении удобно ссылаться на произвольный прочитанный префикс как на u, а строку, продлённую на один символ, обозначать, соответственно, ua.

Прочитав строку u и придя в состояние $(\widehat{q}, f)$, DFA читает символ a и переходит в новое состояние вида $(f'(\widehat{q}), f')$, где значение функции f' для каждого состояния $q_1 \in Q$ определяется так. Пусть 2DFA начал работу на продлённой строке ua на её последнем символе a в состоянии q_1.

- Тогда, если переход в q_1 по a перемещает головку направо, то есть $\delta(q_1, a) = (q'_1, +1)$, то значение $f'(q_1)$ определяется как q'_1.
- Если же переход ведёт налево, то есть $\delta(q_1, a) = (q'_1, -1)$, то автомат перейдёт к последнему символу u в состоянии q'_1. О вычислении, начинающегося с этого места, известно, что через какое-то время оно перейдёт обратно на символ a в состоянии $q_2 = f(q'_1)$. Далее рассматривается переход в состоянии q_2 по a, и если он ведёт к перемещению

головки направо в состоянии q'_2, то значение $f'(q_1)$ можно определить как q'_2; если же головка идёт налево, то продолжение вычисления выясняется далее по тому же образцу.

Так определяется последовательность состояний $q_1, \ldots, q_k, \ldots$, для $k \geqslant 0$, где $\delta(q_i, a) = (q'_i, -1)$ и $q_{i+1} = f(q'_i)$ для всех $i \in \{1, \ldots, k\}$. Есть три возможности: во-первых, эта последовательность может быть бесконечной, что означает, что 2DFA зацикливается, возвращаясь к символу a вновь и вновь в тех же состояниях; в этом случае значение $f(q_1)$ не определено. Во-вторых, последовательность может закончиться, натолкнувшись на неопределённое значение f или на неопределённый переход $\delta(q_i, a)$; тогда $f(q_1)$ также остаётся неопределённым. В-третьих, автомат может однажды придти на a в очередном состоянии q_{k+1}, после чего перейти направо по переходу $\delta(q_{k+1}, a) = (q'_{k+1}, +1)$; и тогда $f'(q_1)$ определяется как q'_{k+1}.

Первый компонент пары определяется применением полученной функции f' к состоянию $f'(\widehat{q})$. Если это значение остаётся неопределённым, то DFA переходит в мёртвое состояние, в котором дочитывает строку до конца и отвергает (и это единственное использование мёртвого состояния).

Принимающие состояния DFA определяются подобно значениям функции f' в точке $\widehat{q}$ — но проще. А именно, для состояния $(\widehat{q}, f)$ одностороннего автомата сперва рассматривается состояние $q_1 = \widehat{q}$ исходного автомата, в котором он впервые приходит к метке конца $(\dashv)$. Для этого состояния вычисляется последовательность состояний $q_1, \ldots, q_k, \ldots$, где на каждом шаге 2DFA или перемещается с метки конца налево, или натыкается на неопределённый переход, или же приходит в принимающее состояние. Если последовательность заканчивается принимающим состоянием 2DFA, то в DFA состояние $(\widehat{q}, f)$ помечается как принимающее.

Иными словами, если переобозначить функцию переходов 2DFA по метке конца как частичную функцию $\delta_\dashv \colon Q \to Q$, где $\delta_\dashv(q) = \delta(q, \dashv)$, то множество принимающих состояний одностороннего автомата можно записать в следующем виде.

$$F' = \left\{ (\widehat{q}, f) \;\middle|\; (f \circ \delta_\dashv)^k(\widehat{q}) \in F \text{ для некоторого } k \geqslant 0 \right\}$$

Правильность построения доказывается индукцией по длине строки. Приведённое выше описание работы DFA — это набросок такого доказательства.

Подсчёт числа состояний. Приведённое выше множество состояний DFA было построено с некоторым запасом, чтобы его легче было объяснять. Чтобы установить точный вид требуемых состояний, необходимо заметить следующие две вещи. Во-первых, нет нужды использовать частичную функцию f: если значение $f(q)$ по смыслу неопределённое, то его можно перенаправить в состояние $\widehat{q}$, и тогда при определении переходов вместо неопределённого значения образуется бесконечный цикл, с теми же последствиями. Тогда $f \colon Q \to Q$ — это полностью определённая функция. Во-вторых, образ f должен содержать состояние $\widehat{q}$, поскольку оно достигается при переходе с предыдущего символа в основном вычислении 2DFA. Поэтому из общего числа функций n^n можно вычесть $(n-1)^n$ функций, образ которых не содержит этого состояния. Таких пар $(\widehat{q}, f)$ ровно $n(n^n - (n-1)^n)$, и ещё используется одно мёртвое состояние. □

Пример 3. *Построение из теоремы 1, применённое к 2DFA из примера 2 (рис. 7), даёт DFA, приведённый на рис. 10.*

Каждое состояние вида $(\widehat{q}, f)$ представлено картинкой, на которой всякое значение $f(q) = q'$ обозначается стрелкой из состояния q в левом столбце в состояние q' в правом

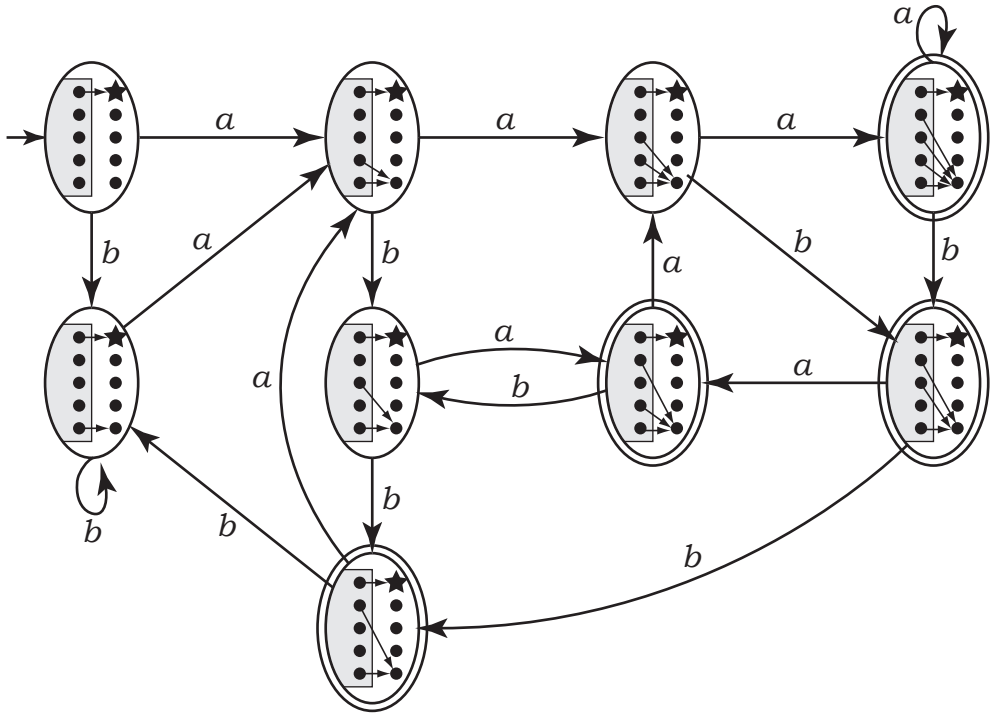


Рис. 10: DFA, построенный по 2DFA для языка $\{a,b\}^* a \{a,b\}^2$.

столбце, а состояние $\widehat{q}$ помечено звёздочкой. Иными словами, в левом столбце приведены состояния, в которых 2DFA начинает вычисление на последнем символе прочитанного участка строки, а в правом столбце — состояния, в которых 2DFA перейдёт направо, покидая уже прочитанный участок.

В этом примере мёртвого состояния нет, потому что у всякой строки есть продолжение, лежащее в языке.

4.1 Преобразование 2DFA к NFA (в лекции не вошло)

Если задачу моделирования работы двустороннего автомата (2DFA) вместо детерминированного автомата (DFA) поручить недетерминированному (NFA), то ему не потребуется запоминать все возможные вычисления — достаточно правильно угадать куски одного-единственного принимающего вычисления и запоминать только их. Оказывается, что чтобы прослеживать ход одного предполагаемого вычисления на строке w, читая её слева направо, не потребуется запоминать произвольную функцию f из Q в Q, и можно ограничиться хранением меньшего объёма сведений о фрагментах предполагаемого вычисления. Немного упрощая, можно сказать, что достаточно хранить *два подмножества* Q — состояния, в которых 2DFA проходит текущий символ слева направо, и состояния, в которых он проходит этот символ справа налево. Точное множество состояний даётся в нижеследующей теореме, приводимой без доказательства.

Теорема 4 (Капуцис [2005])**.** *Для всякого 2DFA с n состояниями существует NFA с $\binom{2n}{n+1}$ состояниями, распознающий тот же язык.*

А сколько это, $\binom{2n}{n+1}$? Это $\frac{(2n)!}{(n-1)!(n+1)!}$, и, если оценить факториалы по формуле Стирлинга, то множители порядка n^n сократятся и останутся только экспоненциальные функции, так что $\binom{2n}{n+1} = \Theta(\frac{1}{\sqrt{n}}4^n)$ — и это существенно меньше, чем n^n.

Капуцис [2005] также показал, что оба построения оптимальны по числу состояний — коль скоро размер алфавита неограничен. Однако, нижняя оценка справедлива лишь для алфавита экспоненциального размера. Для алфавитов меньшего размера оба построения можно немного улучшить: сложность преобразования 2DFA к DFA — до $|\Sigma| \cdot n^{n - \frac{n \ln \ln n}{\ln n} + O(\frac{n}{\ln n})}$, а сложность преобразования к NFA — до $|\Sigma| \cdot \binom{n}{\lfloor n/2 \rfloor} = \Theta(\frac{1}{\sqrt{n}}2^n)$.

Также можно определить двусторонние недетерминированные конечные автоматы (2NFA) с функцией переходов $\delta\colon Q \times (\Sigma \cup \{\vdash, \dashv\}) \to 2^{Q \times \{-1,+1\}}$. Преобразование в NFA работает для них точно так же, как и для 2DFA; преобразование в DFA немного усложняется.

Упражнение 2. *Доказать, что всякий 2NFA с n состояниями можно преобразовать в DFA с 2^{n^2} состояниями.*

4.2 Исследовательские задачи по 2DFA

Сложность преобразования 2NFA к 2DFA — едва ли не самая важная открытая задача теории автоматов. Вопрос в том, можно ли перевести всякий 2NFA с n состояниями в 2DFA с числом состояний, полиномиальном от n — и ответ на него позволит решить одну из самых важных открытых задач в теории сложности вычислений.

Открытый вопрос 1. *Существует ли такой многочлен $p(n)$, что для любого 2NFA с n состояниями есть 2DFA $p(n)$ состояниями, распознающий тот же язык?*

Это действительно трудная задача: её пытались решать многие учёные, в том числе самые знаменитые. Предполагается отрицательный ответ. Однако единственный известный способ преобразования 2NFA к 2DFA — это преобразование 2NFA к DFA, дающее порядка 2^{n^2} состояний, а единственная известная нижняя оценка — всего лишь $\Omega(n^2)$.

Упражнение 3. *Доказать, что длина кратчайшей строки, принимаемой 2DFA с n состояниями, не превосходит 4^n.*

Упражнение 4. *Для всякого n построить такой 2DFA с n состояниями, что кратчайшая принимаемая им строка будет иметь длину, экспоненциальную от n.*

Наилучший известный результат — кратчайшие строки длины порядка 2^n. Поэтому наибольшая возможная длина кратчайшей строки лежит где-то между 2^n и 4^n — но точнее никто пока не знает.

Открытый вопрос 2. *Определить наименьшее основание C, для которого наибольшая длина кратчайшей строки, принимаемой 2DFA с n состояниями, будет ограничена функцией $O(C^n)$.*

Список литературы

[2012] F. Bassino, J. David, C. Nicaud, “Average case analysis of Moore’s state minimization algorithm”, *Algorithmica*, 63:1–2 (2012), 509–531.

[1956] E. F. Moore, “Gedanken experiments on sequential machines”, in: C. E. Shannon, J. McCarthy (Eds.), *Automata Studies*, Princeton University Press, 1956, pp. 129–153.

[1971] J. E. Hopcroft, “An $n \log n$ algorithm for minimizing states in a finite automaton”, *Theory of Machines and Computations* (Haifa, Israel, 16–19 August 1971), 189–196.

[2005] C. A. Kapoutsis, “Removing bidirectionality from nondeterministic finite automata”, *Mathematical Foundations of Computer Science* (MFCS 2005, Gdansk, Poland, 29 August–2 September 2005), LNCS 3618, 544–555.

[1958] A. Nerode, “Linear automaton transformations”, *Proceedings of the AMS*, 9:4 (1958), 541–544.

[1959] M. O. Rabin, D. Scott, “Finite automata and their decision problems”, *IBM Journal of Research and Development*, 3:2 (1959), 114–125.

[1959] J. C. Shepherdson, “The reduction of two-way automata to one-way automata”, *IBM Journal of Research and Development*, 3 (1959), 198–200.